

Chapter 3 and 4

Piyush Pant



C# collections

Generic collections

The **System.Collections.Generic** classes help us to create a generic collection. In this, we store type compatible data elements. This collection does not allow us to store different types of elements.

Non Generic collections

In C#, the **System.Collections** classes help us to create a **non-generic** collection.

For this we use **System.Collections** namespace. Using this we can create classes where we can add data elements of multiple data types.

Generic Collections

1. List Class

- The List<T> class is used to store multiple elements of the **same data type** that can be accessed using the indexes. We can add, insert and remove elements inside the list. Moreover, we can dynamically change the size of the list.

2. Stack Class

- The Stack<T> class is also generic, which means we store data elements of the same data type.
- In stack, the elements are stored in **LIFO**(Last In First Out) manner. With the help of methods, we can perform operations in stack:
- Push()- insert elements
- Pop()- remove elements

3. Queue Class

- In Queue<T> class, the objects are stored in **FIFO**(First In First Out) manner. Here, the elements are inserted at one end and removed from the other. It is implemented using a circular queue. We can perform operations using methods like:
- Enqueue()- add elements
- Dequeue() - remove elements

4. SortedList Class

- The SortedList<TKey,TValue> is an array that consists of key/value pairs that are sorted in an order. Every key inside SortedList<TKey,TValue> must be unique, and using those keys we can access the elements inside it. null value is not accepted as a key.

Generic Collections

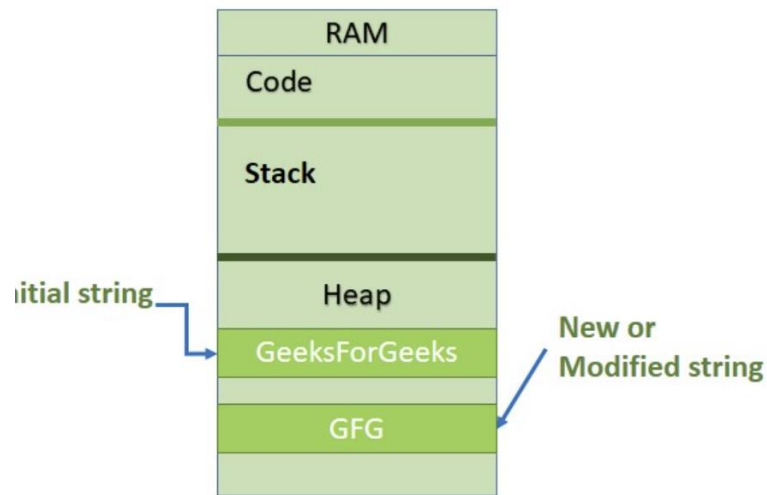
1. ArrayList Class

- ArrayList is non-generic which means we can store elements of multiple data types.
- We use the ArrayList class to implement the functionality of resizable arrays.
- Duplicate elements are allowed inside ArrayList.
- We can use the sort method to sort the elements inside it.

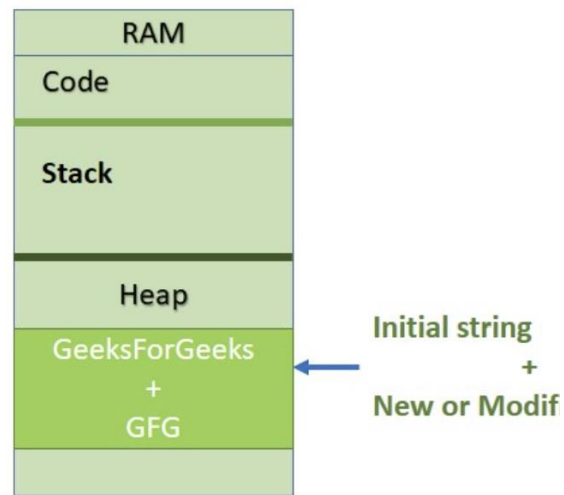
2. Hashtable Class

- Hashtable is also non-generic which consists of key/value pairs that are managed using the hash code of the particular key.
- The key cannot be null however a value can be null.
- We can perform dynamic allocation of the memory inside Hashtable.

String vs StringBuilder



Memory allocation for String



Memory allocation for StringBuilder

- StringBuilder is a dynamic object used to handle mutable strings in C#.
- Unlike the immutable String class which creates a new object after every modification,
- StringBuilder updates the existing buffer and expands its memory as needed.
- This makes it suitable for scenarios involving frequent concatenation, insertion or removal operations.

String vs StringBuilder

Features	String	StringBuilder
Mutability	String are immutable(creates new objects on modification)	StringBuilder are mutable(modifies in place)
Thread-Safe	It is thread-safe	It is not thread-safe
Performance	It is slow because it creates an object each time	It is faster (no object creation)
use Case	Fixed, unchanging strings	Single-threaded string manipulation

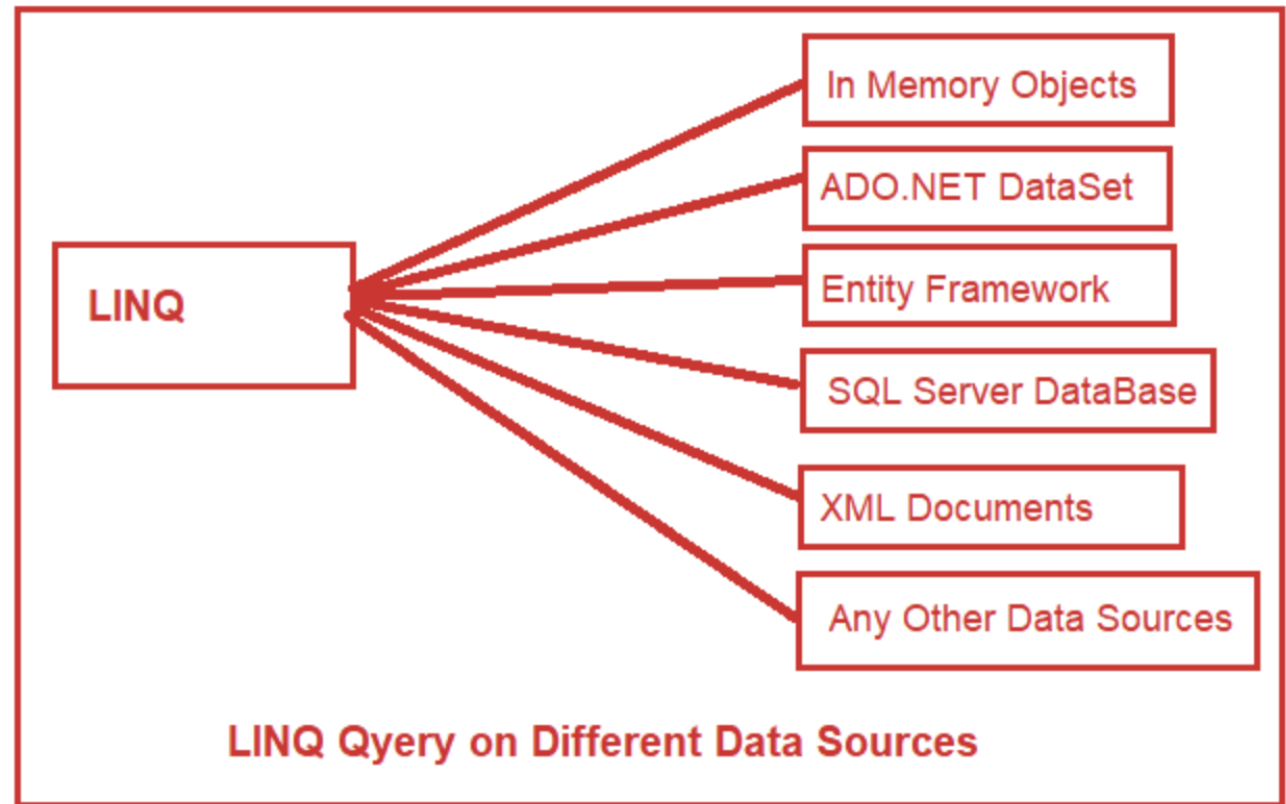


LINQ

- LINQ stands for Language Integrated Query, a Microsoft .NET Framework that provides a standardized way to query data from various data sources using a common syntax within programming languages like C# or VB.
 - LINQ allows developers to write queries to retrieve, manipulate, and transform data from different data sources, such as databases, collections, XML, and In-Memory objects.
 - It was introduced with .NET Framework 3.5 & Visual Studio 2008
-

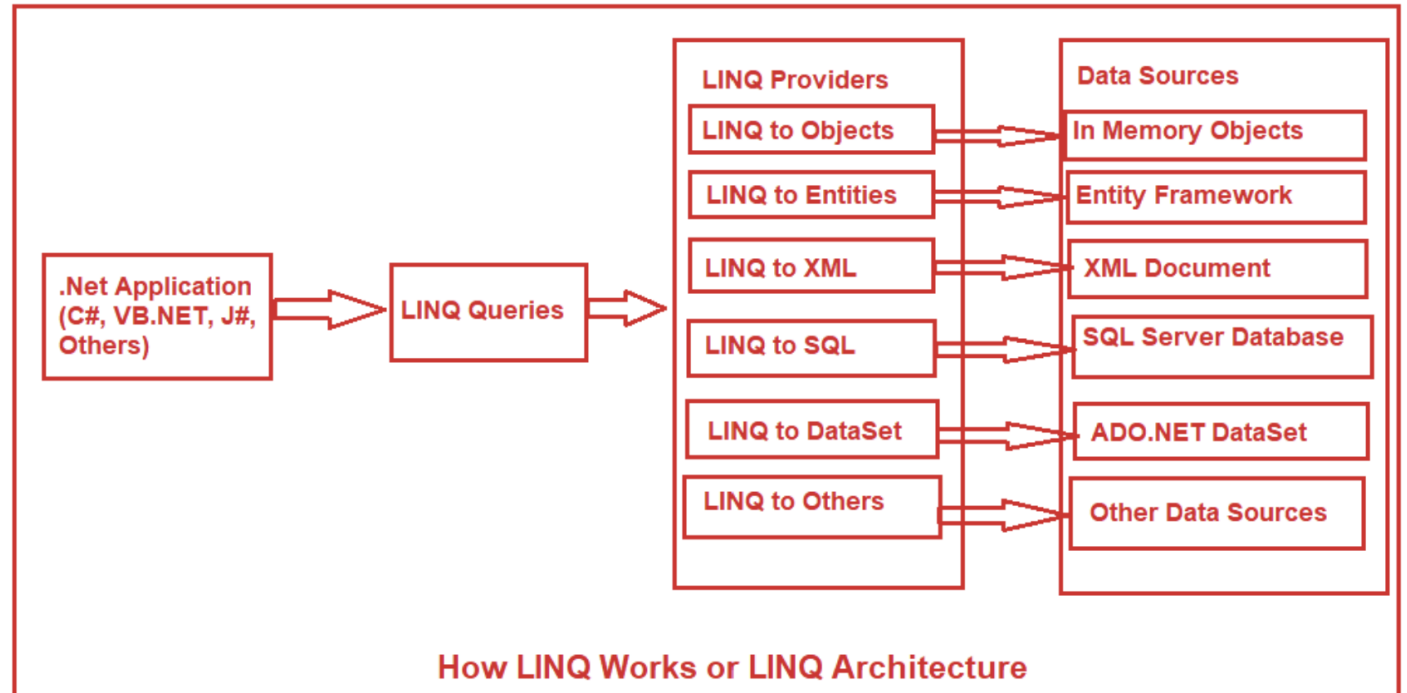
LINQ

- LINQ provides a **Uniform Programming Model** (i.e., Common Query Syntax), which allows us to work with different data sources such as databases, XML Documents, in-memory objects, etc., but using a standard or, you can say, unified coding style. As a result, we are not required to learn different syntaxes to query different data sources



LINQ ARCHITECTURE

- The beauty of LINQ is its ability to provide a consistent querying experience across different types of data sources, but it achieves this through using various LINQ providers.
- Each provider is designed to work with a particular type of data source or a particular way of accessing data.



Ways of writing LINQ Queries

Query Syntax: It is similar to SQL and is often more readable for those familiar with SQL. It starts with a from clause followed by a range variable and includes standard query operations like where, select, group, join, etc.

```
var query = from c in customers           //initialization
            where c.City == "London"      //condition
            select c.Name;                //selection
```

Method Syntax (Fluent Syntax): It uses extension methods and lambda expressions. It can be more concise and is preferred when writing complex queries because it can be easier to read and compose.

```
var query = customers.Where(c => c.City == "London").Select(c => c.Name);
//iniitialization , condition ,selection
```